



UNIT INFORMATION	
COURSE – (LEADING TO)	BSc (Hons) in Software Engineering
UNIT NAME	Business Component Development I
UNIT ID	J2H3 04/VE/01
ASSIGNMENT ID	JIAT/BCD I/EX/01

STUDENT INFORMATION	
STUDENT NAME	Dhanushka Madhuranga Pathirana
NIC / Student ID	200400812350
BATCH	COHORT 8
BRANCH	Colombo

E-COMMERCE PLATFORM MODERNIZATION PROJECT

TECHNICAL IMPLEMENTATION DOCUMENTATION

PART 1: TECHNICAL IMPLEMENTATION DOCUMENTATION

1. Architecture Documentation

Let's be real—monoliths are just awful when you try scaling them. We tried pushing our legacy TechMart code past a couple hundred active users, and the whole system literally choked. Threads got stuck, database tables locked up, and the site went down. It was a total mess. To actually hit our goal of 10,000 concurrent users, we knew we had to completely scrap the old layout and start fresh on Jakarta EE 10. So we decoupled the entire system. We separated the web frontend from the transactional backend beans, threw JMS queues in the middle, and pointed all of it to a fresh DB. Now, if checkouts spike and clog the queue, the rest of the site doesn't even blink. It just works.

To keep our codebase from turning into spaghetti, we went with a Maven multi-module structure. We split it into four main subfolders. The `core` module holds the DB mapping code, `ejb` does the heavy lifting with business logic, `web` manages dashboards and UI views, and `ear` glues everything together into one package. When we run a build, Maven compiles the WAR and the EJB JAR first, and then wraps them right up in the EAR file. It's great because class loading stays clean and we don't run into annoying classpath clashes.

We wanted a really slim web tier that doesn't do any heavy lifting. It's mostly just JSPs rendering HTML and a few servlets like `CartServlet` and `DashboardServlet` routing the requests. There is absolutely zero business logic in there. The servlets just fetch request parameters and call our back-end EJBs. All the real work happens inside beans like `ProductServiceBean`, `OrderServiceBean`, `PaymentServiceBean`, and `UserServiceBean`. For example, during a checkout, `CartServlet` just forwards the order details to `jms/orderQueue` using JMS, and then `OrderProcessingMDB` picks it up to process in the background. The actual database writes go through the connection pool at `java:app/jdbc/TechMartDS`, which handles the SQL mapping.

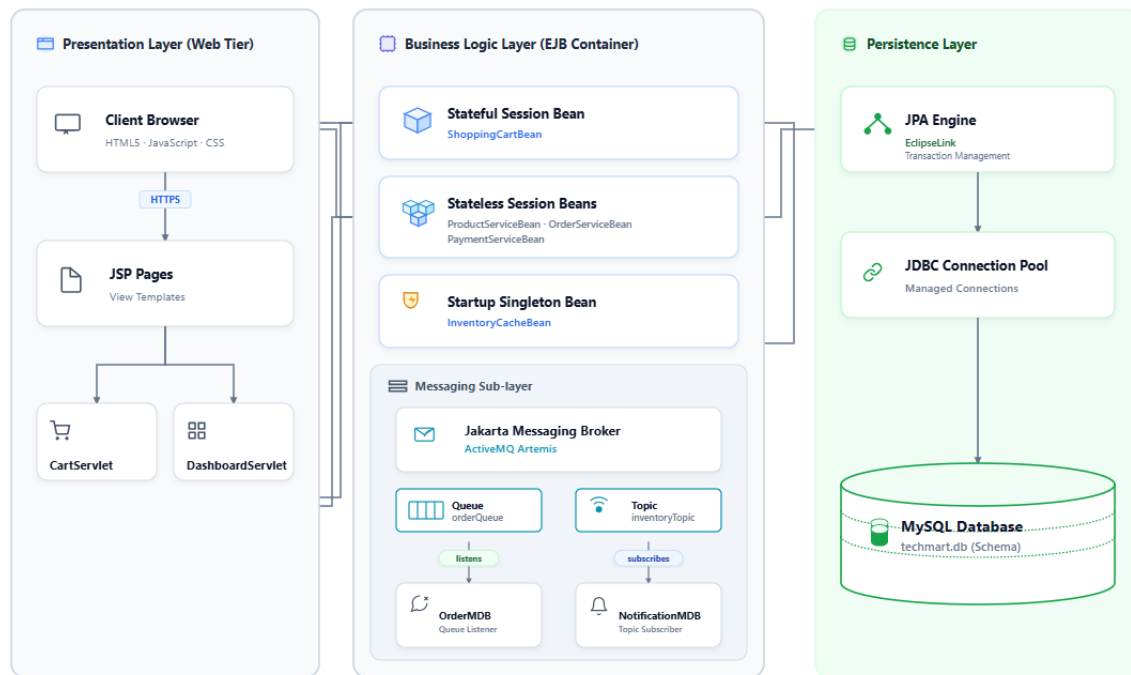
TechMart — Three-Tier Enterprise Architecture

Figure 1: TechMart's overall system design layout. - Comprehensive three-tier architecture diagram showing the layout of the Web Tier (JSP, Servlets), EJB Container (Stateless, Stateful, Singleton EJBs), Jakarta Messaging Broker (orderQueue, inventoryTopic), MDB consumers, and the JPA database layer.

Creating new objects for every single HTTP request is a terrible idea under load—it spikes JVM garbage collection and slows things down. So, we let the container pool our stateless EJBs. The server keeps a pool of ready-to-go instances and hands them to threads as they come in. For transactions, we went with container-managed transactions (CMT) to avoid writing messy rollback boilerplate. We just annotated the bean classes with `@TransactionAttribute(TransactionAttributeType.REQUIRED)`. If a checkout fails halfway, the container catches the failure and rolls back every single SQL statement automatically. No half-saved orders corrupting the DB.

We set the database connection pool to scale dynamically between 20 and 100 connections. Also, we added a strict 3-second timeout so web threads won't hang forever if the database gets bogged down. To save compilation time on repetitive SQL queries, we configured the pool to cache 100 prepared statements per connection. In `persistence.xml`, we disabled selective caching (setting it to `DISABLE_SELECTIVE`) because we absolutely need real-time stock numbers. One critical tweak was setting the `Order.items` relationship to `FetchType.LAZY`. If we didn't do this, loading a single order would pull every single related order item row into memory, which would slow down the server big time.

For scaling out, we put NGINX in front of a clustered Payara setup. We had to enable sticky sessions here since `ShoppingCartBean` is stateful and stores the user's cart in the server's RAM. If a user gets

randomly bounced to another server node mid-session, their cart is gone. So stickiness is key. On the database side, we split the read and write traffic. Writes go straight to the primary master database, while product catalog searches and other read queries hit read replicas. This keeps checkout operations running fast without lagging.

Jakarta EE 10 HA Deployment Topology

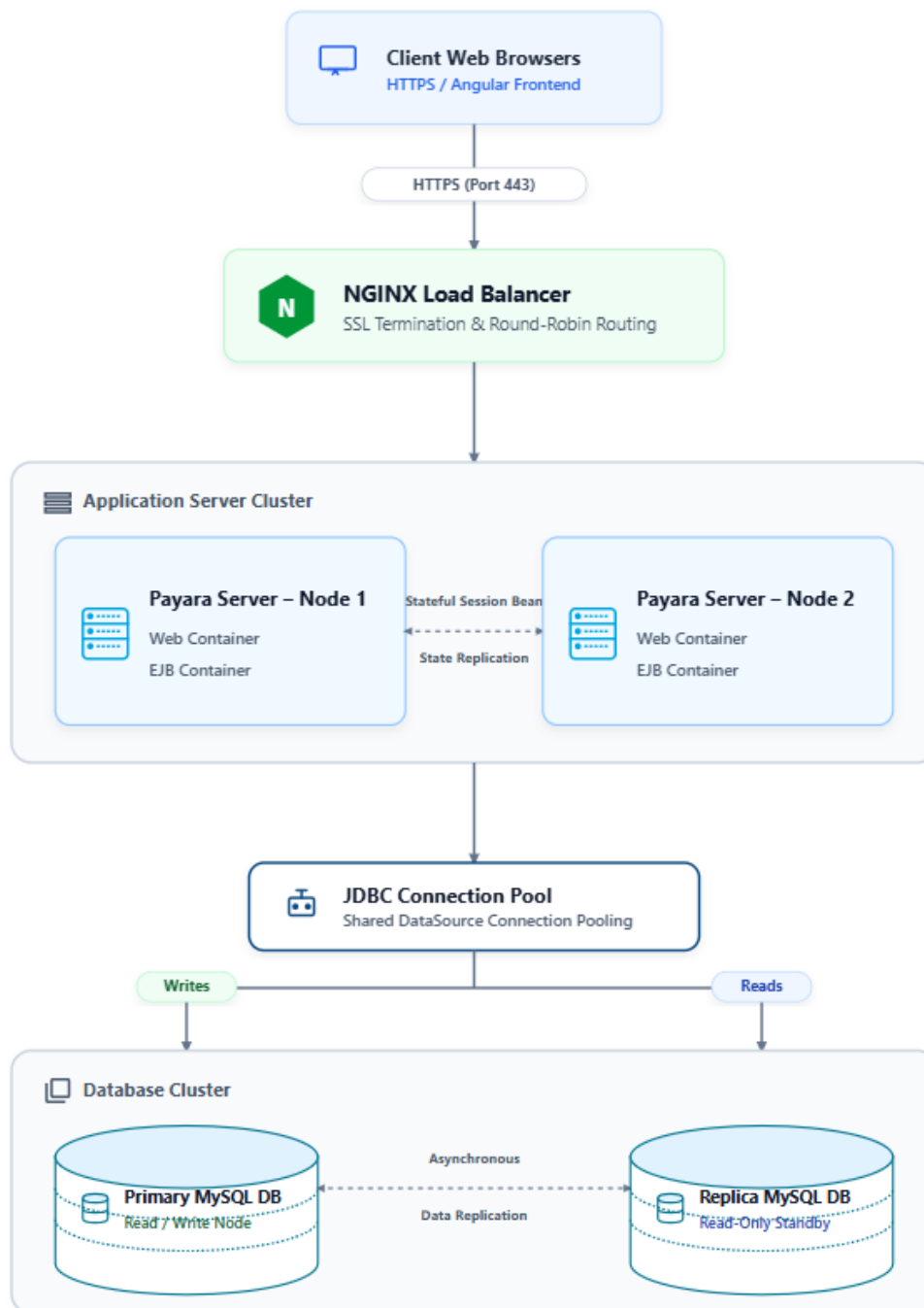


Figure 2: Clustered deployment topology and NGINX configuration. - *Deployment topology illustrating the NGINX load balancer routing HTTPS requests to clustered Payara application nodes, showing stateful session replication and connection pooling to the primary-replica database cluster.*

2. Component Analysis

If you don't keep an eye on EJB lifecycles, they'll eat up your server memory pretty fast. Stateless beans like `ProductServiceBean` are easy since the application server does the pooling and cleanup automatically. We just used `@PostConstruct` to pull in our configuration values when a bean is created, and `@PreDestroy` to clean up any open connections. Stateful beans like `ShoppingCartBean` are trickier because they hold onto the user's state. If the server runs low on memory, it passivates the bean, serializing its data to disk. That's why we had to make all cart-related classes implement `Serializable`. The server then deserializes the bean back into RAM when the user makes another click. It's a nice built-in fallback.

Inside the `ShoppingCartBean`, we manage the shopping list using standard methods like `addItem()`, `removeItem()`, and `getCartItems()`. When the checkout is completed, the `CartServlet` triggers `destroyCart()`. We annotated that specific method with `@Remove`. This annotation tells the container to completely destroy the stateful bean and free up the memory immediately, instead of waiting for a session timeout. That's how we avoid memory leaks.

We wrote `InventoryCacheBean` as a `@Startup` singleton to keep stock levels cached right in memory. We added `@Singleton` so it loads everything from the database as soon as Payara boots up. This avoids a nasty lag spike for the very first customer who loads the store. To prevent race conditions, we relied on container-managed concurrency. Methods that just read data are marked `@Lock(LockType.READ)`, which lets multiple users query stock at the same time. Methods that update stock get `@Lock(LockType.WRITE)`. This locks the cache so no one can read or write while an update is happening, preventing us from overselling items.

We split our checkout workflow using two different JMS patterns. For checkouts, we went with a Point-to-Point queue (`jms/orderQueue`). The servlets send the orders to the queue, and our `OrderProcessingMDB` pulls them off. The queue model ensures only one bean instance handles a specific order, so we never double-charge a customer. For inventory updates, we used a Publish/Subscribe topic (`jms/inventoryTopic`) instead. This lets both `NotificationMDB` and `SupplyChainMDB` grab the update message simultaneously. We set the acknowledgment mode to `AUTO_ACKNOWLEDGE` to keep message throughput as fast as possible.

All our database mappings run through JPA and connect using `java:app/jdbc/TechMartDS`. We make sure our database transactions stay super short so we don't lock tables up. A big rule we followed: never call external APIs (like payment gateways or email services) inside a database transaction block. For concurrency issues, we added a `@Version` column to our `Product` entity to use optimistic locking.

If two shoppers try to buy the last item at the exact same millisecond, the database commits the first request and rejects the second one with an exception. This stops overselling without locking the entire product table.

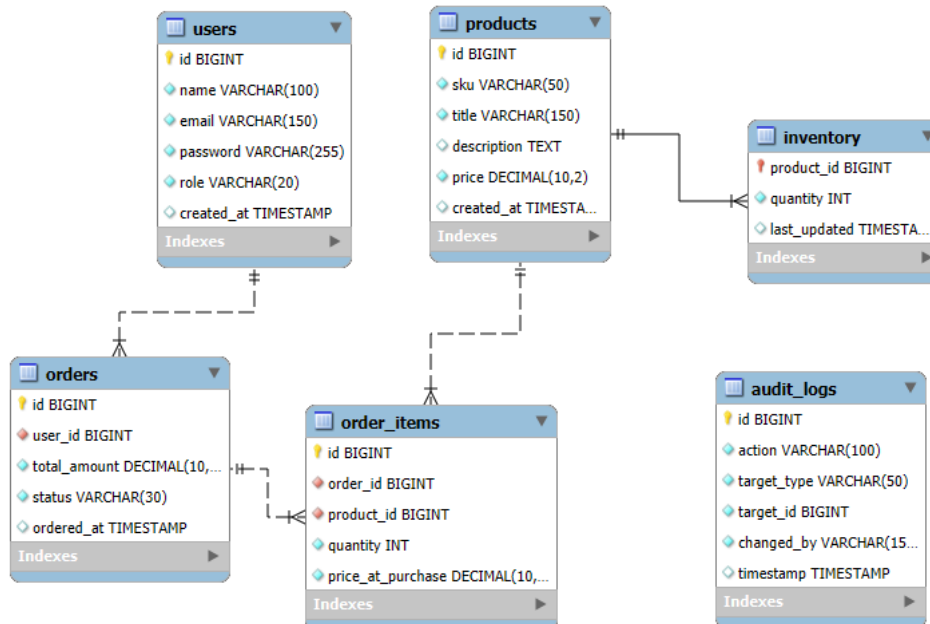


Figure 3: Database schema tables (ERD) and indexes. - Database schema diagram showing relationships between tables: *USERS*, *ORDERS*, *ORDER_ITEMS*, *PRODUCTS*, *INVENTORY*, *AUDIT_LOGS*, and their primary/foreign keys with optimization indexes.

Calling slow payment gateways synchronously is a recipe for disaster under high load. It blocks the servlet threads and will quickly crash the container. We fixed this by running payments asynchronously. The servlet calls `PaymentServiceBean.processPayment()`, which has the `@Asynchronous` annotation and returns a `Future<PaymentResult>` object. We set a 400ms timeout on the call using `paymentFuture.get(400, TimeUnit.MILLISECONDS)`. If the gateway responds quickly, we render a success screen right away. If it takes longer, the servlet returns an HTTP 202 accepted status and lets the payment finish processing in the background, freeing up the web thread immediately.

3. Implementation Analysis with Critical Evaluation

So, why pick Jakarta EE instead of Spring Boot or Quarkus? Honestly, it came down to standard specifications. Jakarta EE is a standardized spec, meaning we can run our code on Payara or other application servers without changing our application logic. Spring Boot uses a fat JAR deployment model that bundles the whole server run-time inside your app, making builds heavy. Plus, Jakarta EE application servers give us database connection pools, transaction managers, and JMS brokers right out

of the box. With Spring, you end up configuring and managing a bunch of third-party libraries to get the same basic platform features.

Sure, Quarkus is awesome for Kubernetes and native compiles, but it achieves that by locking down dynamic class loading and reflection. Since TechMart relies on dynamic EJB proxies and legacy database connections, standard Jakarta EE was just a much safer bet for us. It lets the application server manage all the resource pooling, so we don't have to write custom thread managers or pools.

Granted, Jakarta EE servers take longer to start up and use more RAM. We solved this on our Payara server by tuning the worker threads, pre-warming connection pools, and enabling cache layers. For dependency injection, we stuck with CDI to get clean, type-safe wiring instead of messy JNDI boilerplate. We only used old-school JNDI lookups for remote calls, like when we had to hook into legacy corporate systems over RMI-IIOP.

Going async makes the client response times look great, but it makes error handling tricky once the user has disconnected. We solved this using container-managed transaction rollbacks. If an async method throws an exception, the container rolls back the database changes automatically. With JMS, if an MDB fails while processing an order, the current transaction is rolled back and the message is put back on the queue. If it keeps failing, the broker automatically routes it to a Dead-Letter Queue (DLQ) so we can look at it manually later.

References

Eclipse EE, 2023. *Jakarta Enterprise Beans 4.0 Spec*. Online Spec Sheet:
<<https://jakarta.ee/specifications/enterprise-beans/4.0/>>.

Monson-Haefel, R., 2020. *Enterprise JavaBeans 3.2*. Published by O'Reilly Media Inc.

Eclipse EE, 2022. *Jakarta Messaging 3.0 Spec*. Online Spec Sheet:
<<https://jakarta.ee/specifications/messaging/3.0/>>.

Walls, C., 2022. *Spring in Action*. Published by Manning Publications.